

The Queue ADT: Two Ends, and a Circle

CSCI 235 — Class #10

Tong Yi

Hunter College, CUNY

Thursday, October 1, 2026

A Summary of Where We Have Been

As promised, the running summary.

Class	Topic	The idea to keep
1–3	C++ review, classes, memory	Dynamically allocated memory must eventually be released.
4–5	MiniVector	Growing an array means allocating a larger one and copying the existing elements.
6	Abstract Data Types	The contract is separate from the implementation.
7	Nodes and linked lists	A chain you can only enter at head.
8	Insert, delete, reverse	To change the next pointer that leads to a node, hold the node before it.
9	Stacks	LIFO. Push and pop at the front give $O(1)$.
10 (today)	Queues	FIFO. Add at one end, remove at the other.

Tuesday ended on one sentence: a stack touches one end, a queue touches two. Today is that sentence, paid for in full.

Your Questions from Tuesday

Thank you for the slips. Four questions came up, and they are all good ones:

- 1 **What is Valgrind?** And will I have to write a Valgrind report on a test?
- 2 **Linked or array-based: how do I choose?**
- 3 **Bracket matcher:** are we comparing the top and the *bottom* of the stack?
- 4 **Can we go over the array stack again?**

A few of you also listed topics you want to see again: the stack, more on ADTs, pointers, and reversing a linked list. The last slide of this section says where each one is covered.

Q1: What Is Valgrind?

Think of a **building inspector**. You build the program; Valgrind *runs* it and reports the invalid memory accesses and memory leaks that it detects.

```
==1582== Invalid write of size 8
==1582==   at 0x10C548: Queue::enqueue(int) (Queue.cpp:66)
==1582==   by 0x10AF28: main (Ex5Demo.cpp:83)
==1582== Address 0x4e27258 is 8 bytes inside a block of size 16 free'd
==1582==   at 0x484A61D: operator delete(void*, unsigned long)
==1582==   by 0x10C5C5: Queue::dequeue() (Queue.cpp:90)
```

Read it from the top:

Q1: What Is Valgrind?

Think of a **building inspector**. You build the program; Valgrind *runs* it and reports the invalid memory accesses and memory leaks that it detects.

```
==1582== Invalid write of size 8
==1582==   at 0x10C548: Queue::enqueue(int) (Queue.cpp:66)
==1582==   by 0x10AF28: main (Ex5Demo.cpp:83)
==1582== Address 0x4e27258 is 8 bytes inside a block of size 16 free'd
==1582==   at 0x484A61D: operator delete(void*, unsigned long)
==1582==   by 0x10C5C5: Queue::dequeue() (Queue.cpp:90)
```

Read it from the top:

- **What:** an invalid write — we wrote to memory we did not own.

Q1: What Is Valgrind?

Think of a **building inspector**. You build the program; Valgrind *runs* it and reports the invalid memory accesses and memory leaks that it detects.

```
==1582== Invalid write of size 8
==1582==   at 0x10C548: Queue::enqueue(int) (Queue.cpp:66)
==1582==   by 0x10AF28: main (Ex5Demo.cpp:83)
==1582== Address 0x4e27258 is 8 bytes inside a block of size 16 free'd
==1582==   at 0x484A61D: operator delete(void*, unsigned long)
==1582==   by 0x10C5C5: Queue::dequeue() (Queue.cpp:90)
```

Read it from the top:

- **What:** an invalid write — we wrote to memory we did not own.
- **Where:** enqueue, Queue.cpp line 66.

Q1: What Is Valgrind?

Think of a **building inspector**. You build the program; Valgrind *runs* it and reports the invalid memory accesses and memory leaks that it detects.

```
==1582== Invalid write of size 8
==1582==   at 0x10C548: Queue::enqueue(int) (Queue.cpp:66)
==1582==   by 0x10AF28: main (Ex5Demo.cpp:83)
==1582== Address 0x4e27258 is 8 bytes inside a block of size 16 free'd
==1582==   at 0x484A61D: operator delete(void*, unsigned long)
==1582==   by 0x10C5C5: Queue::dequeue() (Queue.cpp:90)
```

Read it from the top:

- **What:** an invalid write — we wrote to memory we did not own.
- **Where:** enqueue, Queue.cpp line 66.
- **Why:** that memory was already free'd, by the delete in dequeue, Queue.cpp line 90.

Q1: What Is Valgrind?

Think of a **building inspector**. You build the program; Valgrind *runs* it and reports the invalid memory accesses and memory leaks that it detects.

```
==1582== Invalid write of size 8
==1582==   at 0x10C548: Queue::enqueue(int) (Queue.cpp:66)
==1582==   by 0x10AF28: main (Ex5Demo.cpp:83)
==1582== Address 0x4e27258 is 8 bytes inside a block of size 16 free'd
==1582==   at 0x484A61D: operator delete(void*, unsigned long)
==1582==   by 0x10C5C5: Queue::dequeue() (Queue.cpp:90)
```

Read it from the top:

- **What:** an invalid write — we wrote to memory we did not own.
- **Where:** enqueue, Queue.cpp line 66.
- **Why:** that memory was already free'd, by the delete in dequeue, Queue.cpp line 90.

This is the stale `back_` bug from today

The failure shows up in one function; the cause is in another. Valgrind's report names *both*.

Q1 (cont.): The Two Lines the Report Points At

The *original* buggy Queue.cpp (dequeue() forgot to reset back_), with the line numbers Valgrind printed.

enqueue() — Valgrind: **invalid write, line 66**

```
60 bool Queue::enqueue(int value) {
61     Node* newNode = new Node{value, nullptr};
62     if (back_ == nullptr) {
63         front_ = newNode;
64         back_ = newNode;
65     } else {
66         back_->next = newNode;
67         back_ = newNode;
68     }
69     size_++;
70     return true;
71 }
```

dequeue() — Valgrind: **freed here, line 90**

```
84 bool Queue::dequeue() {
85     if (front_ == nullptr) {
86         return false;
87     }
88     Node* toDelete = front_;
89     front_ = front_->next;
90     delete toDelete;
91     size--;
92     return true;
93 }
```

Q1 (cont.): The Two Lines the Report Points At

The *original* buggy Queue.cpp (dequeue() forgot to reset back_), with the line numbers Valgrind printed.

enqueue() — Valgrind: **invalid write, line 66**

```
60 bool Queue::enqueue(int value) {
61     Node* newNode = new Node{value, nullptr};
62     if (back_ == nullptr) {
63         front_ = newNode;
64         back_ = newNode;
65     } else {
66         back_->next = newNode;
67         back_ = newNode;
68     }
69     size_++;
70     return true;
71 }
```

dequeue() — Valgrind: **freed here, line 90**

```
84 bool Queue::dequeue() {
85     if (front_ == nullptr) {
86         return false;
87     }
88     Node* toDelete = front_;
89     front_ = front_>next;
90     delete toDelete;
91     size--;
92     return true;
93 }
```

- Line 90 deletes the only node, but back_ still holds its address.

Q1 (cont.): The Two Lines the Report Points At

The *original* buggy Queue.cpp (dequeue() forgot to reset back_), with the line numbers Valgrind printed.

enqueue() — Valgrind: **invalid write, line 66**

```
60 bool Queue::enqueue(int value) {
61     Node* newNode = new Node{value, nullptr};
62     if (back_ == nullptr) {
63         front_ = newNode;
64         back_ = newNode;
65     } else {
66         back_>next = newNode;
67         back_ = newNode;
68     }
69     size_++;
70     return true;
71 }
```

dequeue() — Valgrind: **freed here, line 90**

```
84 bool Queue::dequeue() {
85     if (front_ == nullptr) {
86         return false;
87     }
88     Node* toDelete = front_;
89     front_ = front_>next;
90     delete toDelete;
91     size--;
92     return true;
93 }
```

- Line 90 deletes the only node, but back_ still holds its address.
- Line 66 later follows that stale back_ and writes to freed memory.

Q1 (cont.): The Two Lines the Report Points At

The *original* buggy Queue.cpp (dequeue() forgot to reset back_), with the line numbers Valgrind printed.

enqueue() — Valgrind: **invalid write, line 66**

```
60 bool Queue::enqueue(int value) {
61     Node* newNode = new Node{value, nullptr};
62     if (back_ == nullptr) {
63         front_ = newNode;
64         back_ = newNode;
65     } else {
66         back_->next = newNode;
67         back_ = newNode;
68     }
69     size_++;
70     return true;
71 }
```

dequeue() — Valgrind: **freed here, line 90**

```
84 bool Queue::dequeue() {
85     if (front_ == nullptr) {
86         return false;
87     }
88     Node* toDelete = front_;
89     front_ = front_->next;
90     delete toDelete;
91     size_--;
92     return true;
93 }
```

- Line 90 deletes the only node, but back_ still holds its address.
- Line 66 later follows that stale back_ and writes to freed memory.

The missing line, before delete toDelete;
if (front_ == nullptr) { back_ = nullptr; }

Q1 (cont.): How to Run Valgrind

Two commands, typed in the terminal in your project folder:

Step 1: build the program (the executable)

```
g++ -std=c++17 -g Stack.cpp Queue.cpp Ex5Demo.cpp -o ex5demo
```

Step 2: run it under Valgrind

```
valgrind --leak-check=full ./ex5demo
```

Q1 (cont.): How to Run Valgrind

Two commands, typed in the terminal in your project folder:

Step 1: build the program (the executable)

```
g++ -std=c++17 -g Stack.cpp Queue.cpp Ex5Demo.cpp -o ex5demo
```

Step 2: run it under Valgrind

```
valgrind --leak-check=full ./ex5demo
```

- **ex5demo is the executable file:** the program that g++ builds from your .cpp files. -o ex5demo chooses that name.

Q1 (cont.): How to Run Valgrind

Two commands, typed in the terminal in your project folder:

Step 1: build the program (the executable)

```
g++ -std=c++17 -g Stack.cpp Queue.cpp Ex5Demo.cpp -o ex5demo
```

Step 2: run it under Valgrind

```
valgrind --leak-check=full ./ex5demo
```

- **ex5demo is the executable file:** the program that g++ builds from your .cpp files. `-o ex5demo` chooses that name.
- Valgrind does not read your source code. It *runs* `./ex5demo` and monitors its memory use for errors.

Q1 (cont.): How to Run Valgrind

Two commands, typed in the terminal in your project folder:

Step 1: build the program (the executable)

```
g++ -std=c++17 -g Stack.cpp Queue.cpp Ex5Demo.cpp -o ex5demo
```

Step 2: run it under Valgrind

```
valgrind --leak-check=full ./ex5demo
```

- **ex5demo is the executable file:** the program that g++ builds from your .cpp files. `-o ex5demo` chooses that name.
- Valgrind does not read your source code. It *runs* `./ex5demo` and monitors its memory use for errors.
- `-g` includes debugging information, so a report can show source lines such as `Queue.cpp:66`.

Q1 (cont.): How to Run Valgrind

Two commands, typed in the terminal in your project folder:

Step 1: build the program (the executable)

```
g++ -std=c++17 -g Stack.cpp Queue.cpp Ex5Demo.cpp -o ex5demo
```

Step 2: run it under Valgrind

```
valgrind --leak-check=full ./ex5demo
```

- **ex5demo is the executable file:** the program that g++ builds from your .cpp files. `-o ex5demo` chooses that name.
- Valgrind does not read your source code. It *runs* `./ex5demo` and monitors its memory use for errors.
- `-g` includes debugging information, so a report can show source lines such as `Queue.cpp:66`.
- A clean run ends with `All heap blocks were freed and 0 errors.`

Q1 (cont.): Will a Test Ask for a Report?

No. Valgrind is *not* tested on the midterm or the final, so you will never have to write a report from memory.

Why we teach it	Professional practice: before you deliver any program, you check it for leaks.
Where it counts	Your projects and exercises . The memory-safety points (15 on Exercise 5) come from running Valgrind on your code.
What a test <i>can</i> ask	The skill Valgrind automates: read a short piece of code and find the memory mistake by hand.

A test-style question (on today's slip)

`enqueue(1); dequeue(); enqueue(2);` — this short sequence breaks a queue whose `dequeue()` forgot one line. Which line, and what goes wrong? You answer in words: no tool output needed.

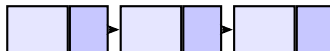
Q2: Linked or Array-Based — How Do I Choose?

Array: a parking lot



fixed number of numbered spots

Linked: train cars



couple on a car when needed

Ask yourself. . .

Lean toward

Do I know a maximum size?

array

(fixed capacity)

Could it need to grow beyond a fixed capacity?

linked

(one new each)

Is memory per item tight?

array

(no pointers)

Which one am I more comfortable writing?

your call

(both are correct)

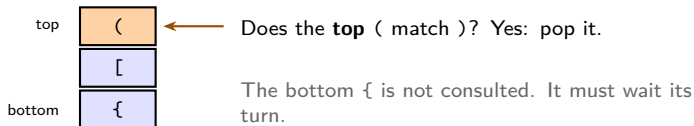
Which mistakes could you handle better? **Circular array:** getting the wrap-around index wrong with `%`. **Linked:** a *stale* pointer, meaning a pointer that still holds the address of memory you already deleted, like `back_` in today's bug.

Both give $O(1)$ for every operation, so speed does not decide.

Q3: The Bracket Matcher Only Ever Looks at the Top

No comparison with the bottom, ever. The stack's only window is its top, so the top is all you get to compare.

Reading `{[()]}` — just arrived at the first `)`

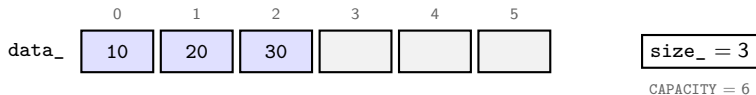


Why the top is the right one

The top is the *most recently opened* bracket that is still unmatched, and that is the only one a closing bracket is allowed to close. In `([ ])`, at the `)` the top is `[`, not `(`. Mismatch: not balanced.

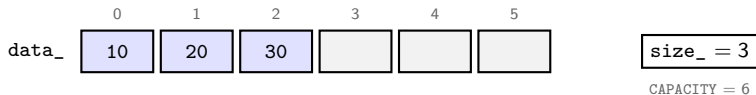
The bottom does matter at the very end: if the stack is not empty after the last character, some bracket was never closed. Empty means balanced.

Q4: The Array Stack — Two Variables



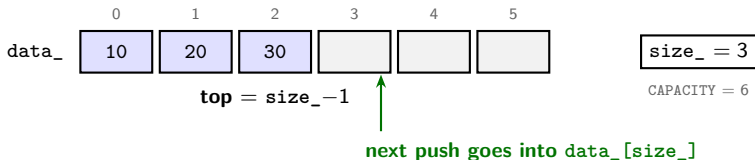
- One array and one count. `data_` is the array: items sit in slots 0, 1, 2, ... in the order pushed.

Q4: The Array Stack — Two Variables



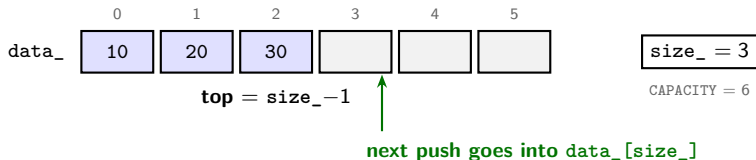
- One array and one count. `data_` is the array: items sit in slots 0, 1, 2, ... in the order pushed.
- `size_` is how many items the stack holds right now.

Q4: The Array Stack — Two Variables



- One array and one count. `data_` is the array: items sit in slots 0, 1, 2, ... in the order pushed.
- `size_` is how many items the stack holds right now.
- The same number tells you where things are: the top is `data_[size_ - 1]`, and the next free slot is `data_[size_]`.

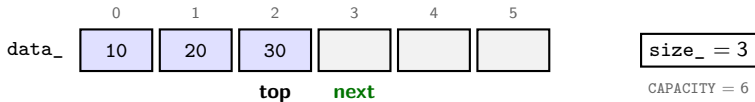
Q4: The Array Stack — Two Variables



- One array and one count. `data_` is the array: items sit in slots 0, 1, 2, ... in the order pushed.
- `size_` is how many items the stack holds right now.
- The same number tells you where things are: the top is `data_[size_ - 1]`, and the next free slot is `data_[size_]`.
- Empty: `size_ = 0`, so there is no top. Full: `size_ = CAPACITY`, so there is no next slot.

Q4: push(value) — Plan First, Then Code

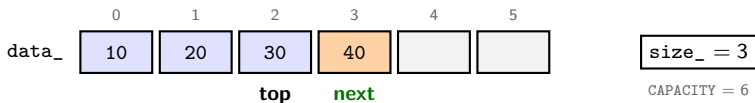
- ❶ **Is there room?** If the array is full (`size_ equals CAPACITY`), return false.



Is it full? `size_ = 3` and `CAPACITY = 6`: no, there is room. We are pushing 40.

Q4: push(value) — Plan First, Then Code

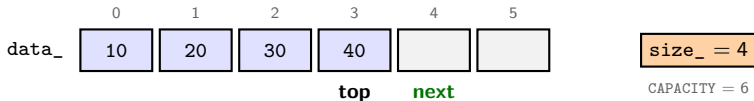
- 1 **Is there room?** If the array is full (`size_ equals CAPACITY`), return false.
- 2 **Store the value** in the next free slot.



Store 40 in `data_[size_]`, which is `data_[3]`. `size_` is still 3, so the stack does not count 40 yet.

Q4: push(value) — Plan First, Then Code

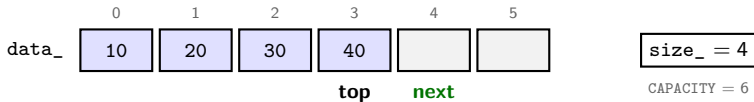
- ❶ **Is there room?** If the array is full (`size_ equals CAPACITY`), return false.
- ❷ **Store the value** in the next free slot.
- ❸ **Count it:** add one to `size_`.



Count it: `size_++` makes `size_ = 4`. Now 40 is the top.

Q4: push(value) — Plan First, Then Code

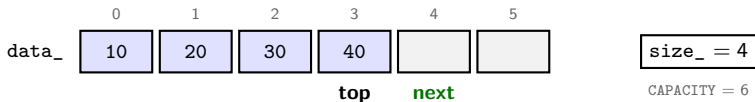
- ❶ **Is there room?** If the array is full (`size_ equals CAPACITY`), return false.
- ❷ **Store the value** in the next free slot.
- ❸ **Count it:** add one to `size_`.
- ❹ **Report success:** return true.



Done: return true. Stack, bottom to top: 10, 20, 30, 40.

Q4: push(value) — Plan First, Then Code

```
bool ArrayStack::push(int value) {  
    if (size_ == CAPACITY) { // 1. no room  
        return false;  
    }  
    data_[size_] = value;    // 2. store in the next free slot  
    size_++;                // 3. count it  
    return true;            // 4. success  
}
```

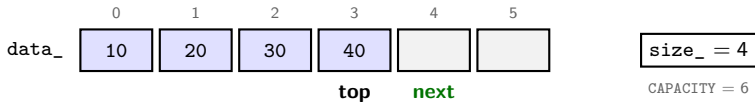


Example: push(40) on a stack holding 10, 20, 30.

Done: return true. Stack, bottom to top: 10, 20, 30, 40.

Q4: `pop()` — Plan First, Then Code

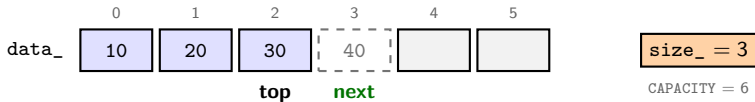
- ❶ **Is there anything to remove?** If the stack is empty (`size_` is 0), return `false`.



Is it empty? `size_ = 4`: no, so there is an item to remove.

Q4: pop() — Plan First, Then Code

- 1 **Is there anything to remove?** If the stack is empty (`size_` is 0), return `false`.
- 2 **Remove the top:** subtract one from `size_`.

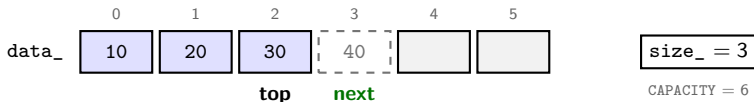


still 40, but outside the stack

`size_ -` makes `size_ = 3`. The top is now `data_[2]`, which holds 30.

Q4: pop() — Plan First, Then Code

- ❶ **Is there anything to remove?** If the stack is empty (`size_` is 0), return `false`.
- ❷ **Remove the top:** subtract one from `size_`.
- ❸ **Report success:** return `true`.

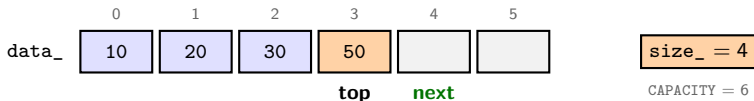


still 40, but outside the stack

return `true`. Look at cell 3: it still holds 40.

Q4: pop() — Plan First, Then Code

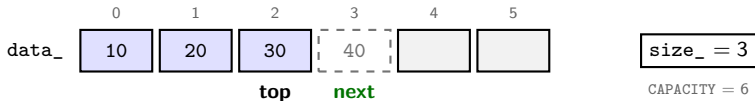
- ❶ **Is there anything to remove?** If the stack is empty (`size_` is 0), return `false`.
- ❷ **Remove the top:** subtract one from `size_`.
- ❸ **Report success:** return `true`.



Why pop() erases nothing: the 40 stays in `data_[3]`, but `size_ = 3` says only slots 0–2 belong to the stack. Erasing would be extra work for no benefit: the next push simply overwrites the slot (here, `push(50)`). An `int` slot always holds *some* number; “empty” just means `size_` does not count it.

Q4: pop() — Plan First, Then Code

```
bool ArrayStack::pop() {  
    if (size_ == 0) {           // 1. nothing to remove  
        return false;  
    }  
    size_--;                   // 2. that's all: data_ is not erased  
    return true;                // 3. success  
}
```

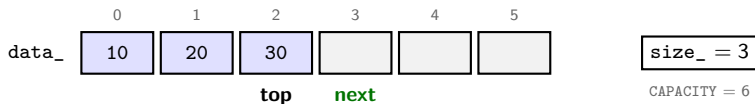


still 40, but outside the stack

Nothing is erased: `size_` alone removes the top.

Q4: `top()` — Plan First, Then Code

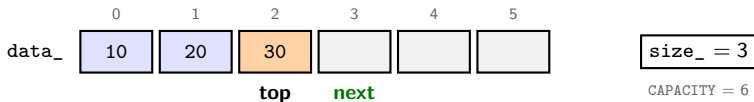
- ① **Precondition:** the stack is not empty (the caller checked `empty()`).



Precondition: the stack is not empty. Here `size_ = 3`, so it is fine. `top()` does not check: the caller must call `empty()` first.

Q4: `top()` — Plan First, Then Code

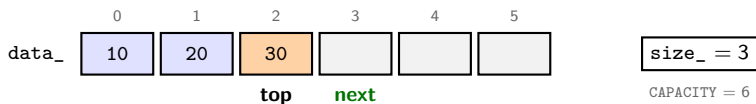
- 1 **Precondition:** the stack is not empty (the caller checked `empty()`).
- 2 **Find the top:** it is the last stored item, at index `size_ - 1`.



The newest item is the last one stored, so its index is `size_ - 1 = 2`.

Q4: `top()` — Plan First, Then Code

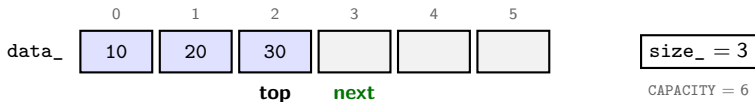
- 1 **Precondition:** the stack is not empty (the caller checked `empty()`).
- 2 **Find the top:** it is the last stored item, at index `size_ - 1`.
- 3 **Return it** without changing `size_`: `top()` only looks, `pop()` removes.



`return data_[2]`, which is 30. Nothing changes: `size_` is still 3 and `data_` is untouched.

Q4: top() — Plan First, Then Code

```
int ArrayStack::top() const {  
    return data_[size_ - 1]; // precondition: !empty()  
}
```



If `size_ = 0`, then `data_[-1]` is outside the array: undefined behavior. That is why the precondition matters.

Also on Your Slips: Where to Look Again

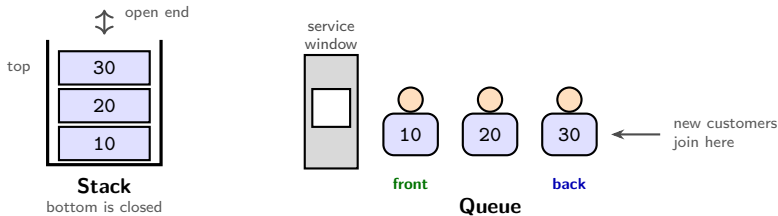
Topic	Where to revisit it
Stack	Class 9 slides, then Exercise 5, Tasks A, B, E . Trace the warm-up table on paper first.
ADTs	“The contract is separate from the implementation”: the Class 6 slides, and today’s <i>Two Implementations, One Contract</i> slide.
Pointers	Classes 7–8. Rule to keep: a pointer changes the field <i>inside the node it points at</i> .
Reversing a linked list	Class 8, and the “Reversing With a Stack” backup slide in Class 9. Exercise 5 Task F (optional) practises the same idea on a queue.
Valgrind	Q1 above.

Still stuck after that? Bring it to office hours, or write it on today’s slip. A slip question gets a written answer in the next class.

Prediction 1: Same Calls, Different Container

You run the same four lines on a **stack** and on a **queue**:

```
add(10); add(20); add(30); remove();
```



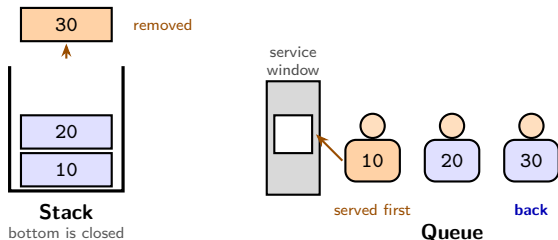
The pictures show each container after the three add calls, just before `remove()`.

Which value does each one remove?

- (A) Stack removes 30; queue removes 30
- (B) Stack removes 30; queue removes 10
- (C) Stack removes 10; queue removes 30
- (D) Both remove 10

Prediction 1: Reveal

(B). The stack takes the newest; the queue takes the oldest.



FIFO: First In, First Out

The queue is the fair one. Nobody who arrived later gets served first — which is exactly why it models a line at the registrar, a print spooler, or a list of tasks waiting their turn.

The Queue ADT: The Whole Contract

Operation	What it promises	If the queue is empty
enqueue(v)	v joins at the back	—
dequeue()	removes the item at the front	invalid operation — behavior depends on the interface
front()	reads the front item, removes nothing	invalid operation — behavior depends on the interface
empty()	true when there is nothing in it	returns true
size()	how many items are waiting	returns 0

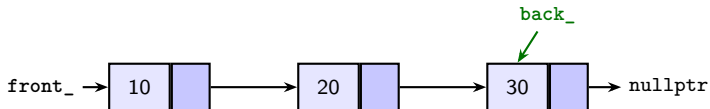
Compare it to Tuesday's slide

Five operations, same shape, same two invalid cases. **One word changed:** *top* became **front** and **back**.

That one word is the entire difficulty of today's class. Adding and removing now happen at **opposite ends** — and neither implementation handles that for free.

Prediction 2: Where the Tail Pointer Finally Pays

On Tuesday a `tail_` pointer was *useless* for a stack: it reached the last node, but not the node before it.



A queue adds at the back and removes at the front. With both pointers, which operations are $O(1)$?

- (A) Both enqueue and dequeue
- (B) enqueue only — dequeue must walk
- (C) dequeue only — enqueue must walk
- (D) Neither

Prediction 2: Reveal

(A) **Both.** And the reason is worth one minute.

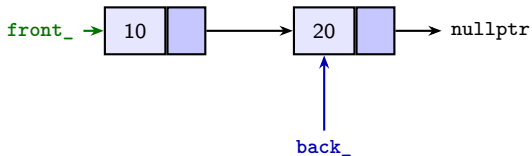
	Which link changes?	Can you reach it?
enqueue at back	back_→next — the <i>last</i> node's own field	Yes. back_ points at it.
dequeue at front	front_ itself — not a node's field at all	Yes. You hold it.
<i>pop at back (Tue.)</i>	<i>the second-to-last node's next</i>	<i>No. Walk from the front: $O(n)$.</i>

The rule, one last time

A pointer lets you change the field **inside the node it points at** — never the field of the node before it. A queue never needs the node before the back, so a single back_ pointer is enough. A stack-at-the-back does, so it is not.

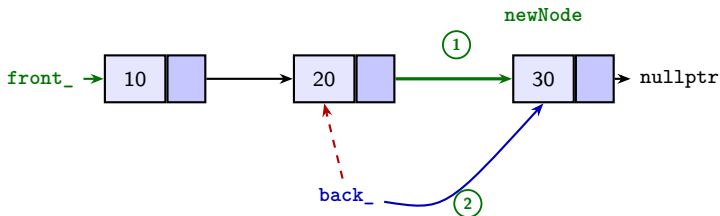
Enqueue: Hook On, Then Advance

Before enqueue(30): the queue holds 10 and 20.



Enqueue: Hook On, Then Advance

After enqueue(30): two pointer writes, in this order.



Why this order

① `back_ -> next = newNode`; — while `back_` still points at the old last node. ② `back_ = newNode`;

Do it the other way and `back_` moves to `newNode` too early. Then `back_ -> next = newNode` makes the new node point to *itself*. The old queue is still reachable from `front_`, but it is no longer connected to `back_`.

You have seen this shape before: it is the classic prepend bug from Class 8, where writing `head = newNode` first made `newNode -> next = head` point the node at itself. Same mistake, different pointer.

The Linked Queue: enqueue() — Your Turn

Two cases. Decide what goes in Step 1 and Step 2 before you press the key.

```
bool LinkedQueue::enqueue(int value) {  
    Node* newNode = new Node{value, nullptr};  
    STEP 1    // what if the queue is empty?  
    STEP 2    // otherwise: hook on, then advance  
    size_++;  
    return true;  
}
```

Hints

Step 1: with no nodes, which pointers must change? **Step 2:** which pointer write comes first, back_->next or back_?

The Linked Queue: enqueue() — Your Turn

Two cases. Decide what goes in Step 1 and Step 2 before you press the key.

```
bool LinkedList::enqueue(int value) {
    Node* newNode = new Node{value, nullptr};
    if (back_ == nullptr) {    // STEP 1: empty queue
        front_ = newNode;      // it is BOTH ends at once
        back_ = newNode;
    }
    STEP 2    // otherwise: hook on, then advance
    size_++;
    return true;
}
```

Step 1 answer

An empty queue has `front_ == back_ == nullptr`, so the new node becomes *both* ends.

The Linked Queue: enqueue() — Your Turn

Two cases. Decide what goes in Step 1 and Step 2 before you press the key.

```
bool LinkedQueue::enqueue(int value) {
    Node* newNode = new Node{value, nullptr};
    if (back_ == nullptr) {    // STEP 1: empty queue
        front_ = newNode;      // it is BOTH ends at once
        back_ = newNode;
    } else {
        back_->next = newNode; // STEP 2: ① hook on
        back_ = newNode;      //           ② then advance
    }
    size_++;
    return true;
}
```

Step 2 answer

Hook the new node onto the *old* last node first ①, and only then move back_ ②.

The Linked Queue: dequeue() — Your Turn

dequeue is Class 8's removeFront, with **one line added**. Find it before you press the key.

```
bool LinkedQueue::dequeue() {  
    if (front_ == nullptr) { return false; }  
    Node* toDelete = front_;  
    front_ = front_>next;  
    STEP 3    // <-- the extra line. why?  
    delete toDelete;  
    size--;  
    return true;  
}
```

A hint for Step 3

Ask what back_ is pointing at after you remove the *last remaining* node.

The Linked Queue: dequeue() — Your Turn

dequeue is Class 8's removeFront, with **one line added**. Find it before you press the key.

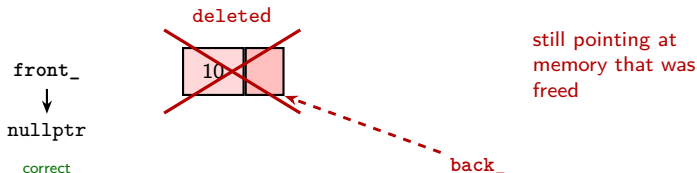
```
bool LinkedQueue::dequeue() {  
    if (front_ == nullptr) { return false; }  
    Node* toDelete = front_;  
    front_ = front_>next;  
    if (front_ == nullptr) { back_ = nullptr; } // STEP 3  
    delete toDelete;  
    size_--;  
    return true;  
}
```

Step 3 answer

When the last node leaves, front_ becomes nullptr. back_ must become nullptr too; otherwise it points at the node you are about to delete.

Step 3 Is the Bug Everyone Writes

Remove the last node **without** resetting back_:



What happens next

`empty()` says true — it checks `front_`, which is correct. So the queue *looks* fine. Then the next `enqueue` tests `back_ == nullptr`, finds it is *not*, and writes `back_>next = newNode;` **into freed memory**. The failure may happen nowhere near the mistake — which is why Valgrind exists, and why it is part of your grade.

Two Pointers, Four Cases to Get Right

Whenever a structure has two pointers into it, the interesting cases are the ones where they *coincide*.

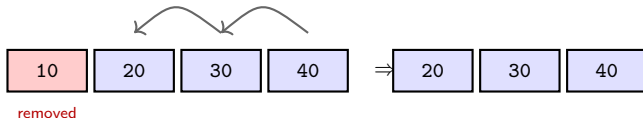
Situation	front_	back_
Empty	nullptr	nullptr
One item	the only node	<i>the same</i> node
Many items	first node	last node
enqueue into empty	must be set	must be set
dequeue the last one	becomes nullptr on its own	must be reset by hand

The pattern to take with you

The empty-to-one and one-to-empty transitions are where two-pointer structures break. **Test those two first** — on Exercise 5, and on every structure like this you ever write.

Prediction 3: The Obvious Array Queue

Keep the items in an array, front at index 0. enqueue writes at the end; dequeue takes `data_[0]` and **shifts everything left**.



What is the cost of dequeue?

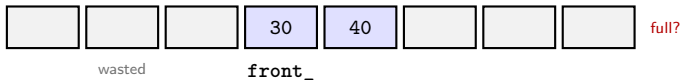
- (A) $O(1)$ — it is just an array read
- (B) $O(n)$ — every remaining item moves
- (C) $O(\log n)$

Prediction 3: Reveal — and the Fix Nobody Guesses

(B) $O(n)$. Which is unacceptable: a queue's whole job is cheap removal at the front.

First idea: don't shift — move the front index instead

Keep `front_` as an *index*. dequeue becomes `front_++`. Now it is $O(1)$.

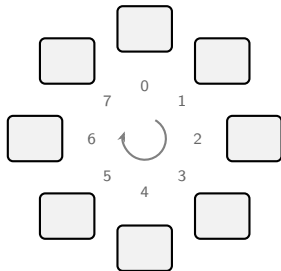


And now the new problem

The queue **creeps to the right**. Enqueue and dequeue a hundred times and you run off the end of the array — while most of it sits empty. You have solved the time problem, but created a space-reuse problem.

The Circular Buffer: Let the Array Wrap

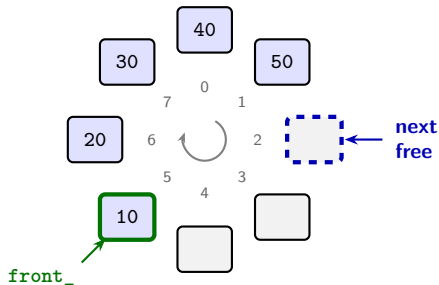
The array does not really end at its last index. Instead, **the slot after the last slot is slot 0 again**, so the array behaves like a ring (or a clock face: after 12 comes 1).



The ring: CAPACITY = 8 slots, numbered 0 to 7. Going forward from slot 7 takes you to slot 0.

The Circular Buffer: Let the Array Wrap

The array does not really end at its last index. Instead, **the slot after the last slot is slot 0 again**, so the array behaves like a ring (or a clock face: after 12 comes 1).



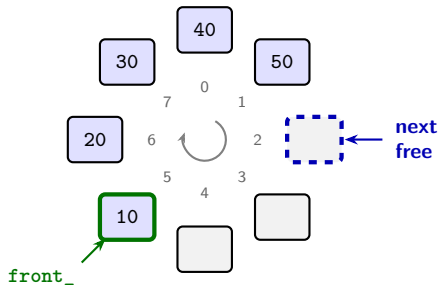
The ring: CAPACITY = 8 slots, numbered 0 to 7. Going forward from slot 7 takes you to slot 0.

The queue: start at `front_` (slot 5) and read forward, wrapping around: 10, 20, 30, 40, 50. Five items in order, with no shifting and nothing wasted.

Next free is the first slot after the last item: slot 2.

The Circular Buffer: Let the Array Wrap

The array does not really end at its last index. Instead, **the slot after the last slot is slot 0 again**, so the array behaves like a ring (or a clock face: after 12 comes 1).



The ring: CAPACITY = 8 slots, numbered 0 to 7. Going forward from slot 7 takes you to slot 0.

The queue: start at `front_` (slot 5) and read forward, wrapping around: 10, 20, 30, 40, 50. Five items in order, with no shifting and nothing wasted.

Next free is the first slot after the last item: slot 2.

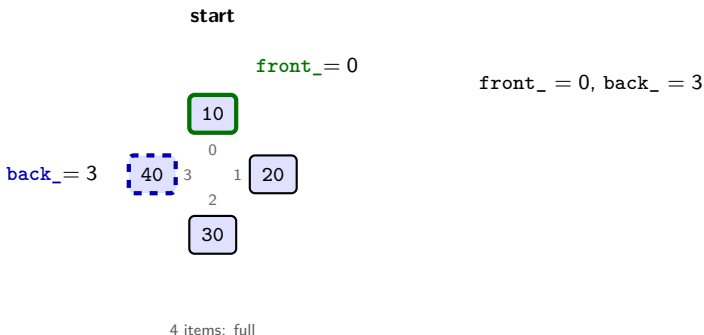
Moving one slot forward. If i is the index of the slot you are standing on, the next slot is $(i + 1) \% \text{CAPACITY}$.

$i = 3: (3 + 1) \bmod 8 = 4.$

$i = 7: (7 + 1) \bmod 8 = 0$ (the wrap).

Deque Down to Empty: Watch front_

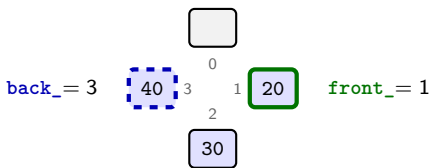
CAPACITY = 4. Start with 10, 20, 30, 40 in the queue. While the queue is nonempty, front_ identifies the first item and back_ the last. In this attempted two-index representation, dequeue moves only front_.



Deque Down to Empty: Watch front_

CAPACITY = 4. Start with 10, 20, 30, 40 in the queue. While the queue is nonempty, front_ identifies the first item and back_ the last. In this attempted two-index representation, dequeue moves only front_.

dequeue 10



front_ = 0, back_ = 3

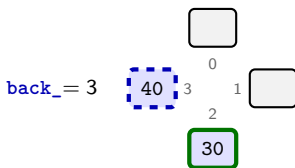
remove 10: front_ → 1

3 items

Deque Down to Empty: Watch front_

CAPACITY = 4. Start with 10, 20, 30, 40 in the queue. While the queue is nonempty, front_ identifies the first item and back_ the last. In this attempted two-index representation, dequeue moves only front_.

dequeue 20



front_ = 2

2 items

front_ = 0, back_ = 3

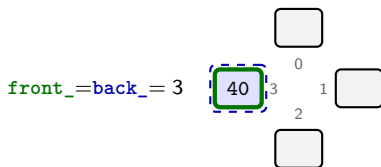
remove 10: front_ → 1

remove 20: front_ → 2

Deque Down to Empty: Watch front_

CAPACITY = 4. Start with 10, 20, 30, 40 in the queue. While the queue is nonempty, front_ identifies the first item and back_ the last. In this attempted two-index representation, dequeue moves only front_.

dequeue 30



front_ = 0, back_ = 3

remove 10: front_ → 1

remove 20: front_ → 2

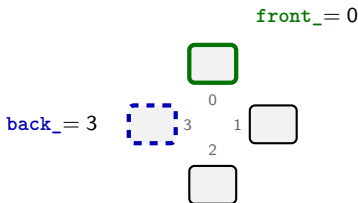
remove 30: front_ → 3

1 item: front_ == back_
Now dequeue 40: what should happen?

Dequeuing Down to Empty: Watch front_

CAPACITY = 4. Start with 10, 20, 30, 40 in the queue. While the queue is nonempty, `front_` identifies the first item and `back_` the last. In this attempted two-index representation, dequeue moves only `front_`.

dequeue 40



0 items: empty

`front_ = 0, back_ = 3`

remove 10: `front_ → 1`

remove 20: `front_ → 2`

remove 30: `front_ → 3`

remove 40: `front_ → 0 (wraps)`

`back_` never moved, so after the last dequeue the two stored indices are still `front_ = 0` and `back_ = 3` — even though the queue is now empty. Those are exactly the index values we had when the queue was *full*.

Prediction 4: Empty or Full?

The trace you just saw ended with no items left, yet `front_ = 0` and `back_ = 3` — the very same indices the *full* queue had. `CAPACITY = 4`.

Empty and full look identical. How do you tell them apart?

- (A) Check whether `data_[front_]` holds a value
- (B) Not with two ordinary indices — you need extra state or a convention
- (C) Compare `front_` to `CAPACITY`

Prediction 4: Empty or Full?

The trace you just saw ended with no items left, yet `front_ = 0` and `back_ = 3` — the very same indices the *full* queue had. `CAPACITY = 4`.

The test `(back_ + 1) % CAPACITY == front_` gives $(3 + 1) \bmod 4 = 0 = \text{front_}$, which is true for *both* states. (`front_ == back_` would mean exactly one item.)

Empty and full look identical. How do you tell them apart?

- (A) Check whether `data_[front_]` holds a value
- (B) Not with two ordinary indices — you need extra state or a convention
- (C) Compare `front_` to `CAPACITY`

Prediction 4: Reveal

(B) You cannot, with only two ordinary array indices. You need some additional state or convention.

Why (A) fails

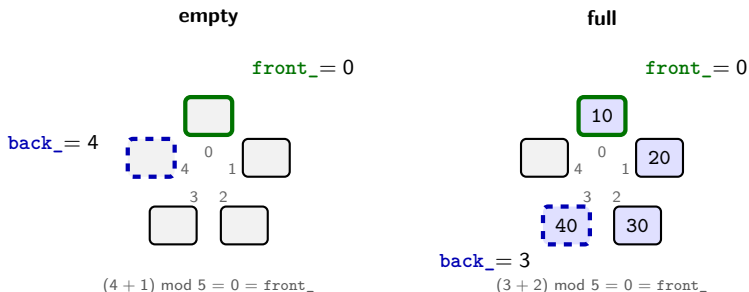
“Holds a value” is not a question you can ask an `int`. An `int` has no special “empty” value. Even after an item is dequeued, its old value may remain in the array, so looking at `data_[front_]` cannot tell you whether that slot currently belongs to the queue. There is no such thing as an empty `int`.

Keep a <code>size_count</code>	<code>empty()</code> is <code>size_ == 0</code> ; <code>full</code> is <code>size_ == CAPACITY</code> . Uses one extra <code>int</code> , and <code>size()</code> is free.
Leave one slot empty	Never let the queue use the last slot: <code>empty</code> is <code>(back_ + 1) % CAPACITY == front_</code> , <code>full</code> is <code>(back_ + 2) % CAPACITY == front_</code> . Costs one slot and makes <code>size()</code> arithmetic

“The linked queue reset `back_` to `nullptr`. Why not reset `back_` here?” It needs a value no real index can have, such as `-1` (see the backup slides). Next: the second row of the table, drawn.

Leave One Slot Empty: What It Looks Like

Here $CAPACITY = 5$, so the queue holds at most 4 items. $back_$ is still the last item.



- **Empty:** $(back_ + 1) \% CAPACITY == front_$.
- **Full:** $(back_ + 2) \% CAPACITY == front_$. Slot 4 is deliberately left empty: 5 slots hold only 4 items.

What we use instead

Keep $front_$ and $size_$. Empty is $size_ == 0$, full is $size_ == CAPACITY$, and every slot is usable. It is also easier to write, and $size()$ comes for free.

The Circular Queue: the Class

Notice there is no member for the back of the queue. It is *computed* when needed.

```
class ArrayQueue {
public:
    bool enqueue(int value);
    bool dequeue();
    int front() const;      // precondition: !empty()
    bool empty() const { return size_ == 0; }
    int size() const { return size_; }
private:
    static const int CAPACITY = 100;
    int data_[CAPACITY];
    int front_ = 0;
    int size_ = 0;
};
```

Two data members besides the array: `front_` is where `dequeue()` removes, and `size_` is how many items are stored.

The Circular Queue: enqueue()

The new item goes into the first free slot, found from `front_` and `size_`.

```
bool ArrayQueue::enqueue(int value) {  
    if (size_ == CAPACITY) { return false; }  
    int insertIndex = (front_ + size_) % CAPACITY;  
    data_[insertIndex] = value;  
    size_++;  
    return true;  
}
```

- **Full?** `size_ == CAPACITY`: return false.
- **Where?** `insertIndex` is the next free slot; the next slide shows why.
- **Store, then count:** write the value, then `size_++`.

The Circular Queue: `dequeue()` and `front()`

`dequeue()` only moves `front_` forward, wrapping around. Nothing is erased.

```
bool ArrayQueue::dequeue() {
    if (size_ == 0) { return false; }
    front_ = (front_ + 1) % CAPACITY;
    size_--;
    return true;
}

int ArrayQueue::front() const {
    return data_[front_]; // precondition: !empty()
}
```

- **Empty?** `size_ == 0`: return false.
- **Step forward:** `(front_ + 1) % CAPACITY` moves to the next slot, and from the last slot back to slot 0.
- **Count it:** `size_--`. The old value stays in `data_`, but `size_` no

Why $(\text{front_} + \text{size_}) \% \text{CAPACITY}$ Is the Next Free Slot

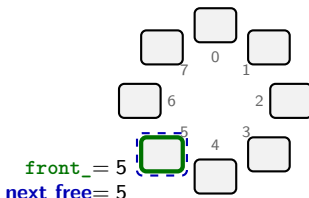
Walk size_ steps forward from front_ , wrapping around: that lands on the first *free* slot. $\text{CAPACITY} = 8$, $\text{front_} = 5$.

1 $\text{size_} = 0:$
 $(5 + 0) \bmod 8 = 5$

2 $\text{size_} = 3:$
 $(5 + 3) \bmod 8 = 0$

3 $\text{size_} = 5:$
 $(5 + 5) \bmod 8 = 2$

4 $\text{size_} = 8:$
 $(5 + 8) \bmod 8 = 5$



Empty: the next item goes at slot 5, where front_ is.

Why $(\text{front_} + \text{size_}) \% \text{CAPACITY}$ Is the Next Free Slot

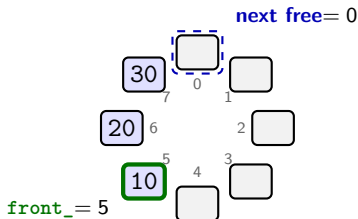
Walk size_ steps forward from front_ , wrapping around: that lands on the first *free* slot. $\text{CAPACITY} = 8$, $\text{front_} = 5$.

1 $\text{size_} = 0$:
 $(5 + 0) \bmod 8 = 5$

2 $\text{size_} = 3$:
 $(5 + 3) \bmod 8 = 0$

3 $\text{size_} = 5$:
 $(5 + 5) \bmod 8 = 2$

4 $\text{size_} = 8$:
 $(5 + 8) \bmod 8 = 5$



Slots 5, 6, 7 are used, so the index *wraps* to slot 0.

Why $(\text{front_} + \text{size_}) \% \text{CAPACITY}$ Is the Next Free Slot

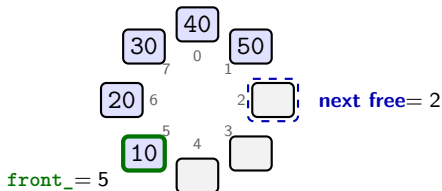
Walk size_ steps forward from front_ , wrapping around: that lands on the first *free* slot. $\text{CAPACITY} = 8$, $\text{front_} = 5$.

1 $\text{size_} = 0$:
 $(5 + 0) \bmod 8 = 5$

2 $\text{size_} = 3$:
 $(5 + 3) \bmod 8 = 0$

3 $\text{size_} = 5$:
 $(5 + 5) \bmod 8 = 2$

4 $\text{size_} = 8$:
 $(5 + 8) \bmod 8 = 5$



Slots 5, 6, 7, 0, 1 are used; the next free slot is 2.

Why $(\text{front_} + \text{size_}) \% \text{CAPACITY}$ Is the Next Free Slot

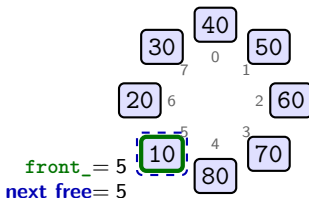
Walk size_ steps forward from front_ , wrapping around: that lands on the first free slot. $\text{CAPACITY} = 8$, $\text{front_} = 5$.

1 $\text{size_} = 0$:
 $(5 + 0) \bmod 8 = 5$

2 $\text{size_} = 3$:
 $(5 + 3) \bmod 8 = 0$

3 $\text{size_} = 5$:
 $(5 + 5) \bmod 8 = 2$

4 $\text{size_} = 8$:
 $(5 + 8) \bmod 8 = 5$



Full: every slot is used, and the formula gives front_ again.

The last row is the point

When the queue is full, the computed next-free slot equals front_ again, the same ambiguity as before. It never causes a bug because $\text{size_} == \text{CAPACITY}$ is checked first: a guard makes the ambiguous case unreachable.

Two Implementations, One Contract — Again

	Circular array	Linked
enqueue / dequeue	$O(1)$	$O(1)$
Capacity	fixed at compile time	limited only by memory
Memory per item	just the item	the item <i>and</i> a pointer
Allocation	none, ever	one new per enqueue
Can leak from queue storage itself?	no	yes, without a destructor
Easy to get wrong	the % arithmetic	back_ on the last dequeue

Same conclusion as Tuesday, and it is not a coincidence

Both satisfy the same five-operation contract. Code written against the ADT cannot tell them apart. What differs is **which mistake you are likely to make** — and you should choose knowing that.

Where Queues Show Up

Print spooler	Jobs print in the order they were sent. Nobody jumps the line.
Task scheduling	The operating system gives each waiting process its turn.
Buffering	Data arrives faster than it can be handled, so it waits in order — keystrokes, network packets, a video stream.
Breadth-first search	Class 12, Project 2: explore every cell one step away, <i>then</i> every cell two steps away. The queue is what enforces “one step at a time.”

The pattern under all four

On Tuesday: when you catch yourself saying *most recent*, you have found a stack.
Today: when you catch yourself saying *in the order they arrived*, or *fairly*, you have found a queue.

One Word, Two Different Searches

This is the slide to remember when Project 2 arrives.

	Stack $\Rightarrow$ DFS	Queue $\Rightarrow$ BFS
Takes next	the most recently discovered cell	the earliest discovered cell
Behaves like	one explorer following a corridor to its end	a flood spreading outward evenly
Finds	a path	the shortest path (by number of edges)
Holds	stack of discovered cells	queue of discovered cells

The basic iterative search structure is otherwise the same

Same loop, same visited-marking, same neighbour expansion. **With the same discovery/visited policy, changing the frontier from a stack to a queue changes depth-first exploration into breadth-first exploration.**

That is not a clever trick — it is what choosing a data structure actually means. The container decides the order of exploration.

What's Due

- **Exercise 4** (Linked List Operations) — due **today**.
- **Project 1B** — due **today**.
- **Exercise 5** (Stack & Queue) — **posted today**, due **Thursday, October 8**.
- **Project 2** (Maze Solver) — **will be posted Friday, October 2**. It uses both of this week's structures.

Before Tuesday

Read **Chapter 13** (the ADT queue) and **Chapter 14** (queue implementations). Tuesday is **recursion and the call stack** — which is Tuesday's stack slide, taken seriously.

Key Takeaways

- 1 A queue is **FIFO**: add at the back, remove from the front. Two ends, so we must keep track of both ends somehow.
- 2 A `back_` pointer makes enqueue $O(1)$ *because* the link that changes lives **inside the node it points at**.
- 3 **Reset `back_` when the queue empties.** The damage shows up on the next enqueue, far from the mistake.
- 4 A circular buffer wraps with `% CAPACITY`, and storing `size_` removes the full-versus-empty ambiguity.
- 5 **Stack $\Rightarrow$ DFS, queue $\Rightarrow$ BFS.** Same basic search structure; the container chooses the order of exploration.

Questions?

Please fill out today's lecture slip.

Exercise 5 goes out today, and Project 2 follows on Friday, October 2 — if something on these two classes is still unclear, this is the moment to say so.

Backup Slides

Backup: The Linked Queue — the Class

```
class LinkedQueue {
public:
    LinkedQueue() = default;
    ~LinkedQueue();
    bool enqueue(int value);
    bool dequeue();
    int front() const; // precondition: !empty()
    bool empty() const { return front_ == nullptr; }
    int size() const { return size_; }
private:
    struct Node { int data; Node* next; };
    Node* front_ = nullptr;
    Node* back_ = nullptr;
    int size_ = 0;
};
```

Two pointers and a count. `empty()` asks `front_`, never `back_` — which is exactly why a stale `back_` can hide.

Backup: The Linked Queue — the Methods

```
bool LinkedList::enqueue(int value) {
    Node* newNode = new Node(value, nullptr);
    if (back_ == nullptr) { front_ = newNode; back_ = newNode; }
    else { back_->next = newNode; back_ = newNode; }
    size_++;
    return true;
}

bool LinkedList::dequeue() {
    if (front_ == nullptr) { return false; }
    Node* toDelete = front_;
    front_ = front_->next;
    if (front_ == nullptr) { back_ = nullptr; }
    delete toDelete;
    size_--;
    return true;
}

LinkedList::~LinkedList() {
    while (front_ != nullptr) {
        Node* toDelete = front_;
        front_ = front_->next;
        delete toDelete;
    }
    back_ = nullptr;
    size_ = 0;
}
```

Backup: Leaving One Slot Empty Instead

The alternative from Prediction 4, for completeness.

```
// members: int front_ = 0; int back_ = CAPACITY - 1; (no size_)
// back_ is the LAST item; an empty queue has back_ one step behind front_

bool empty() const { return (back_ + 1) % CAPACITY == front_; }
bool full() const { return (back_ + 2) % CAPACITY == front_; }
int size() const { return (back_ - front_ + 1 + CAPACITY) % CAPACITY; }

bool enqueue(int value) {
    if (full()) { return false; }
    back_ = (back_ + 1) % CAPACITY;
    data_[back_] = value;
    return true;
}
```

The trade

One slot of `CAPACITY` can never be used, and `size()` costs arithmetic instead of being stored. In exchange, there is no counter that could drift out of step with the indices.

You will meet this version in textbooks and in other people's code — recognise it, and check which convention a class uses before you edit it.

Backup: Using -1 to Mean “Empty”

A second alternative: keep `front_` (first item) and `back_` (last item), and reserve the value -1 for “there is no item.”

```
// members: int front_ = -1; int back_ = -1; (no size_)
bool empty() const { return front_ == -1; }
bool full() const { return !empty() && (back_ + 1) % CAPACITY == front_; }
bool enqueue(int value) {
    if (full()) { return false; }
    if (empty()) { front_ = 0; }           // first item: both indices start at 0
    back_ = (back_ + 1) % CAPACITY;       // from -1 this gives 0
    data_[back_] = value;
    return true;
}
bool dequeue() {
    if (empty()) { return false; }
    if (front_ == back_) {                 // that was the LAST item
        front_ = -1; back_ = -1;         // restore the empty state
    } else {
        front_ = (front_ + 1) % CAPACITY;
    }
    return true;
}
```

The trade

Reserving -1 as an empty marker tells empty from full without `size_`: the extra state is encoded in a sentinel. It plays the role of `nullptr` in the linked queue, and not resetting both indices is the same stale-`back_` bug.

Backup: BFS, So You Can See the Queue

The Project 2 shape, with the maze details removed (neighbors, visited, markVisited are placeholders).

```
#include <queue>

struct Coord { int row; int col; };

void bfs(Coord start) {
    std::queue<Coord> q;
    q.push(start);
    markVisited(start);

    while (!q.empty()) {
        Coord c = q.front(); // the EARLIEST discovered cell
        q.pop();

        for (Coord n : neighbors(c)) {
            if (!visited(n)) {
                markVisited(n); // mark when DISCOVERED,
                q.push(n);      // not when removed
            }
        }
    }
}
```

In Project 2 you use the standard library's `std::queue`: `push` is our `enqueue`, `pop` is our `dequeue` (it returns nothing), and `front`, `empty` keep their names. Change it to `std::stack` (top in place of `front`) and this is depth-first search. Nothing else moves.